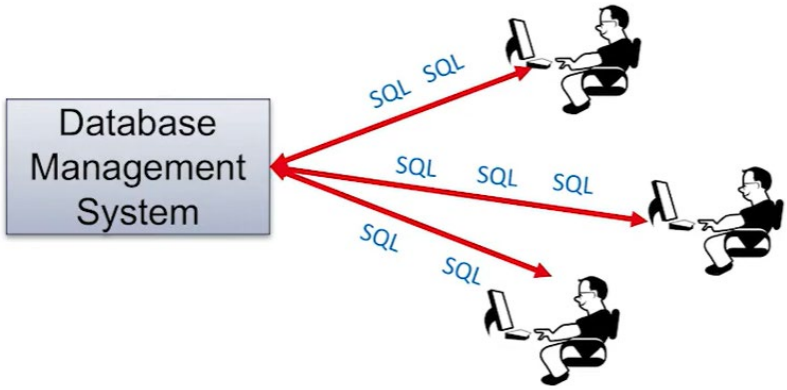
Database Transactions ACID

Virtually any applications you use everday: Lyft, Uber, Expedia, Amazon, Chase, is an application on top of some kind of DBMS. These applications have to run 24/7 all around the world and be reliable.



Applications, when running, always want something to from the DBMS

- Queries and updates
- Real applications are composed of many statements generated by user choices
- Many users work with the applications, and hence the database at the same time



Why Run Transactions Concurrently?

A simple approach to preventing transactions from interfering with one another is to process transactions from all SQL connections one at a time. This works when each transaction is correct on its own, but it can reduce throughput and increase latency.

The database cannot use multiple CPU cores to run different transactions in parallel, and it cannot make progress on another transaction while the current one waits for disk I/O. A short transaction may also have to wait behind a long one, even when they access unrelated data.

A database can make progress on several transactions during the same period. This can improve two measures of performance.

- **Higher throughput.** The database can complete more transactions per second by keeping the CPU and disk busy. Even on a single core, one transaction can use the CPU while another waits for a disk operation. With multiple cores, transactions can also perform CPU work in parallel.
- **Lower latency.** A short transaction can finish while a longer, unrelated transaction is still running. If the transactions compete for the same resources, one may still have to wait.

Throughput measures how much work the database completes per second. Latency measures how long an individual transaction takes to finish.

Example of possible concurrency issue:

Product(id, pname, price)
DollarProducts(id, name, price)

User 1

```
INSERT INTO DollarProducts (name, price)
SELECT pname, price
FROM Product
WHERE price <= 0.99;

DELETE FROM Product
WHERE price <= 0.99;
```

User 2

```
SELECT * FROM DollarProducts;

SELECT * FROM Product;
```

What could go wrong? Inconsistent Reads

ACID Properties of Database Transactions

ACID stands for **Atomicity, Consistency, Isolation, and Durability**. These four properties describe how a database handles a transaction, including what happens when transactions overlap or the system crashes.

Suppose you transfer **$100** from account A, which has **$500**, to account B, which has **$200**.

Property	What it means	How it applies to the transfer
Atomicity	A transaction's changes take effect as a unit.	If the transfer commits, both the debit and credit take effect. If it rolls back, neither takes effect.
Consistency	A correctly written transaction preserves the rules that define valid data.	The transfer should preserve the combined balance of **$700**. If the code mistakenly debits both accounts, it can commit both changes atomically and still violate that business rule.
Isolation	The isolation level controls how concurrent transactions interact and what they can see.	If another connection uses `READ COMMITTED`, it cannot read A's uncommitted **$400** balance. If it uses `READ UNCOMMITTED`, it may see that value in a **dirty read**.
Durability	A transaction's changes survive a crash after the database confirms a durable commit.	If power fails after a durable commit, recovery preserves the new balances of **$400** and **$300**, even if it has to reapply changes from the log.

In SQL, `START TRANSACTION` groups the debit and credit into one transaction. The application checks that both updates succeeded before issuing `COMMIT`. If either update fails, it issues `ROLLBACK`. The isolation level governs what other transactions can see while that work is in progress.

Isolation (Concurrency)

Think about avoiding problems due to concurrency

- If another transaction T2 accesses R and S between steps 4 and 5 of T1, it will see a lower value for R+S.

T1	T2
1. start transaction	
2. read(R)	
3. R = R − 100	
4. write(R)	
	1. start transaction
	2. read(R, S)
	3. print(R+S)
	4. end transaction
5. read(S)	
6. S = S + 100	
7. write(S)	
8. end transaction	

Isolation easy to achieve by running one Xact at a time

- However, recall that serial execution is not desirable

- DBMS interleaves actions of many xacts
    - Actions = reads/writes of DB objects

- DBMS ensures 2 xacts do not "interfere". This is usually achieved by using valid locking protocol.

- Each xact executes as if it ran by itself.
    - Concurrent accesses have no effect on xact's behavior
    - Net effect must be identical to executing all transactions in some serial order
    - Users & programmers think about transactions in isolation
        - Without considering effects of other concurrent Xacts!

Atomicity and Durability

- **A transaction ends in one of two ways:**
    - **Commit** after completing all its actions
        - "commit" is a contract with the caller of the DB
    - **Abort** (or be aborted by the DBMS) after executing some actions
        - Or **system crash** while the xact is in progress; treat as abort.
- **Two key properties** for a transaction
    - **Atomicity:** Either execute all its actions, or none of them
    - **Durability:** The effects of a committed xact must survive failures.
- DBMS typically ensures the above by **logging** all actions:
    - **Undo** the actions of aborted/failed transactions.
    - **Redo** actions of committed transactions not yet propagated to disk when system crashes

The Transaction Manager in SQL

The **transaction manager** is the part of a database management system that tracks transactions and coordinates their completion. SQL gives an application commands such as `START TRANSACTION`, `COMMIT`, and `ROLLBACK`. The database uses those commands to determine which operations belong to the same transaction and how that transaction should end.

SQL command	What it does
`START TRANSACTION`	Starts an explicit transaction.
`COMMIT`	Finalizes the transaction and accepts its changes as a unit.
`ROLLBACK`	Aborts the transaction and discards its uncommitted changes.

Example with the $100 transfer

```
START TRANSACTION;
UPDATE accounts
SET balance = balance - 100
WHERE account_id = 'A';

UPDATE accounts
SET balance = balance + 100
WHERE account_id = 'B';
COMMIT;
```

The transaction manager tracks both `UPDATE` statements as part of **one transaction**. The application should issue `COMMIT` after verifying that both updates succeeded and each affected the intended row. That check matters because an `UPDATE` can succeed as a SQL statement while affecting zero rows. If either update fails or affects zero rows, the application should roll back the transfer.

How it works with concurrency and recovery

- **Concurrency control** governs how the transfer interacts with other transactions. In a system using strict two-phase locking, the transfer holds its exclusive locks on A and B until it commits or aborts. Another transaction that needs a conflicting lock has to wait.
- **Recovery** uses the transaction's commit status to decide which effects to preserve after a crash. In a system using write-ahead logging, the database makes the necessary log records durable before acknowledging a durable commit. Recovery can then redo missing changes from the committed transfer or undo or discard the effects of an incomplete transfer, depending on the database's design.

The transaction boundary matters especially when **autocommit** is enabled. If you run the two updates as separate statements outside an explicit transaction, each can commit independently. An explicit transaction puts the debit and credit under one commit.

Durability means that committed changes survive failures. A backup in secure storage supports durability by giving the database a copy to restore if its primary storage is damaged or lost. The backup restores data up to the point it was taken, and saved transaction logs can help recover changes committed after that point.

Durability also preserves changes after a successful `COMMIT`. For example, after a crash, recovery may **undo** an incomplete transfer and **redo** a committed transfer whose changes hadn't yet reached the data files. Undo supports atomicity, while redo supports durability.

An updated **data page** can be written to disk **before or after `COMMIT`**. It can also stay in memory for a while after the commit.

- **Before `COMMIT`:** The change is still uncommitted, even if the data page is already on disk. If the transaction aborts, the database can undo it.
- **After `COMMIT`:** The updated data page may still be in memory. If the database crashes before writing it, recovery can redo the change using the durable log.

ROLLBACK

Suppose account A has **$500**, but account B is missing. The debit succeeds, then the credit updates **zero rows**. MySQL treats an `UPDATE` that matches zero rows as a successful statement, so the procedure checks `ROW_COUNT()` immediately after each update and raises an error if it didn't update exactly one row. MySQL `ROW_COUNT()` documentation

This example assumes `accounts` is an InnoDB table, `account_id` is unique, and the procedure is called outside an existing transaction.

```
DELIMITER //
CREATE PROCEDURE transfer_100()
BEGIN
    UPDATE accounts
    SET balance = balance - 100
    WHERE account_id = 'A' AND balance >= 100;

    UPDATE accounts
    SET balance = balance + 100
    WHERE account_id = 'B';
END//
DELIMITER ;

CALL transfer_100();
```

```
DELIMITER //
CREATE PROCEDURE transfer_100()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    UPDATE accounts
    SET balance = balance - 100
    WHERE account_id = 'A' AND balance >= 100;

    IF ROW_COUNT() <> 1 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Account A is missing or has insufficient funds';
    END IF;

    UPDATE accounts
    SET balance = balance + 100
    WHERE account_id = 'B';
```

```
    IF ROW_COUNT() <> 1 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Account B is missing';
    END IF;
    COMMIT;
END//
DELIMITER ;

CALL transfer_100();
```

Situation	What happens
A has $500, but B is missing	A is debited $100. The credit updates zero rows, leaving A at $400.
A has only $50, and B exists	The debit updates zero rows because `balance >= 100` is false. B still receives $100.

CONSISTENCY

Consistency is the ACID property that depends most on correct application logic. The database enforces rules you declare, such as primary keys, foreign keys, and `CHECK` constraints. The application must also preserve business rules that the database hasn't been told about.

For example, a $100 transfer should keep the combined balance of the two accounts unchanged. If the SQL mistakenly subtracts $100 from both accounts, the database can commit both updates atomically while the transfer violates that business rule.

Triggers help maintain **consistency** by automatically enforcing business rules when data is inserted, updated, or deleted. For example, a trigger can reject an update that moves an order from `Shipped` back to `Pending`. Because the rule is enforced in the database, it applies whether the update comes from an application or a SQL script. Triggers help preserve the specific rules they're written to enforce.